ME-522: High-Performance Scientific Computing

Project Report

Parallel 2D Steady-State Heat Conduction Solver using Finite Difference Method in Fortran with OpenMP

Name	Harsh Pandey
Roll Number	B23441
Course	ME-522, IIT Mandi
Platform	Dell G16, Intel Core i7-13650HX, 16 GB RAM
Environment	WSL2 Ubuntu 24.04, gfortran 13.3 `-O2 -fopenmp`
GitHub	`https://github.com/Harshpandeyiitian04/ME522-Heat-Solver`
Date	May 4, 2026

Indian Institute of Technology Mandi

Abstract

A parallel two-dimensional steady-state heat conduction solver is implemented in Fortran 90 and parallelised with OpenMP. The 2D Laplace equation is discretised with the five-point Finite Difference Method (FDM) on uniform $N \times N$ grids and solved iteratively by the Jacobi method with Dirichlet boundary conditions ($T = 100\,^{\circ}\mathrm{C}$ on the top edge, $T = 0$ on the remaining three edges). Correctness is verified against a smooth manufactured analytical solution, yielding L_2 refinement ratios of 3.92 and 4.79, confirming $\mathcal{O}(h^2)$ spatial accuracy. All three production grids converge fully to $\varepsilon = 10^{-6}$: $N = 128$ in 31,676 iterations (1.11 s), $N = 256$ in 107,282 iterations (16.2 s), and $N = 512$ in **353,746 iterations** (270 s). Strong-scaling experiments show a best speedup of **4.69**× on 8 threads for $N = 512$ (58.6% efficiency) and 4.36× on 4 threads for $N = 256$ (109%, super-linear, explained by cache effects). The cache-ordering experiment demonstrates a 1.09–1.31× slowdown when the loop order violates Fortran's column-major layout.

Contents

1. Introduction

Under steady-state conditions the temperature $T(x, y)$ in a thin solid plate satisfies the 2D Laplace equation

$$\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} = 0 \quad \text{on } \Omega = [0,1] \times [0,1], \tag{1}$$

subject to Dirichlet boundary conditions. The objectives of this project are:

1. Implement a serial Jacobi-FDM solver in Fortran 90 with proper boundary-condition handling and convergence checking.
2. Verify correctness against a smooth manufactured solution, confirming $\mathcal{O}(h^2)$ spatial accuracy.
3. Parallelise the solver using OpenMP `!$OMP PARALLEL DO` directives with correct treatment of shared and private variables.
4. Conduct a strong-scaling study across grid sizes 128^2, 256^2, 512^2 and thread counts $1, 2, 4, 8$.
5. Investigate the effect of loop ordering on cache performance.

2. Mathematical Formulation

2.1. Boundary conditions

$$T(x,0) = 0, \quad T(0,y) = 0, \quad T(1,y) = 0, \quad T(x,1) = 100\,^{\circ}\mathrm{C}. \tag{2}$$

2.2. FDM stencil

Applying second-order central differences on a uniform grid of spacing $h = 1/(N+1)$ gives the five-point stencil:

$$\boxed{T_{i,j} = \tfrac{1}{4}\big(T_{i-1,j} + T_{i+1,j} + T_{i,j-1} + T_{i,j+1}\big), \quad i,j = 1,\ldots,N.} \tag{3}$$

The local truncation error is $\mathcal{O}(h^2)$.

2.3. Jacobi iteration

$$T_{i,j}^{(k+1)} = \tfrac{1}{4}\big(T_{i-1,j}^{(k)} + T_{i+1,j}^{(k)} + T_{i,j-1}^{(k)} + T_{i,j+1}^{(k)}\big). \tag{4}$$

Because the right-hand side uses only values from iteration k, all N^2 updates per sweep are data-independent, making Jacobi trivially and safely parallelisable. Convergence is declared when

$$\max_{i,j} \big|T_{i,j}^{(k+1)} - T_{i,j}^{(k)}\big| < \varepsilon = 10^{-6}. \tag{5}$$

The spectral radius of the Jacobi iteration matrix for the discrete Laplacian is $\rho = \cos(\pi h) \approx 1 - \frac{\pi^2 h^2}{2}$, giving a theoretical iteration count $k_{\text{conv}} \sim \frac{-2\log\varepsilon}{\pi^2 h^2} \propto N^2$. This explains why $N = 512$ requires 353,746 iterations.

3. Implementation

3.1. Project structure

```
heat2d/
+-- src/
|   +-- heat_serial.f90     serial solver + cache test
|   +-- heat_parallel.f90   OpenMP parallel solver
|   +-- heat_verify.f90     verification (manufactured solution)
+-- scripts/
|   +-- run_scaling.sh      automated scaling study
|   +-- plot_results.py     matplotlib figures
+-- data/                   CSV timing results + field .dat files
+-- plots/                  5 publication PDF figures
+-- report/                 this LaTeX report
+-- Makefile
```

3.2. Serial solver (`heat_serial.f90`)

Two arrays `T(0:N+1, 0:N+1)` and `Tnew(0:N+1, 0:N+1)` are allocated with ghost boundary cells at indices 0 and $N+1$, which are initialised to the Dirichlet values and never overwritten during iteration.

```
do while (err > tol .and. iter < max_iter)
  iter = iter + 1 ;  err = 0.0d0
  do j = 1, N
    do i = 1, N        ! i inner: column-major, cache-friendly
      Tnew(i,j) = 0.25d0*(T(i-1,j)+T(i+1,j) &
                          +T(i,j-1)+T(i,j+1))
      err = max(err, abs(Tnew(i,j)-T(i,j)))
    end do
  end do
  T(1:N, 1:N) = Tnew(1:N, 1:N)  ! BCs preserved at 0,N+1
end do
```

Listing 1: Serial Jacobi loop. `i` is the inner index, aligning with Fortran's column-major storage.

3.3. OpenMP parallel solver (`heat_parallel.f90`)

```
!$OMP PARALLEL DO PRIVATE(i) REDUCTION(max:err) SCHEDULE(static)
do j = 1, N
  do i = 1, N
    Tnew(i,j) = 0.25d0*(T(i-1,j)+T(i+1,j) &
                        +T(i,j-1)+T(i,j+1))
    err = max(err, abs(Tnew(i,j)-T(i,j)))
  end do
end do
!$OMP END PARALLEL DO
```

Listing 2: OpenMP directives on the Jacobi sweep. The outer `j`-loop is distributed across threads.

OpenMP clause justification

- `PRIVATE(i)`: the inner-loop counter must be thread-private; a shared `i` would cause a data race.
- `REDUCTION(max:err)`: each thread accumulates a thread-local maximum residual; the OpenMP runtime combines these at the barrier (available since OpenMP 3.1).
- `SCHEDULE(static)`: contiguous equal-sized row blocks are assigned statically. Since every row does identical work, this is perfectly load-balanced with minimal overhead.
- **No write conflict**: `T` is read-only within the parallel region; threads write to disjoint row chunks of `Tnew`. No critical sections or atomic updates are required.

An identical directive is applied to the array copy (`T = Tnew`) after each sweep to also parallelise the memory-copy bottleneck.

3.4. Build system (Makefile)

```
make serial    gfortran -O2 -Wall -std=f2008
make parallel  gfortran -O2 -Wall -std=f2008 -fopenmp
make verify    build and run heat_verify (confirms O(h^2))
make run       full scaling study via scripts/run_scaling.sh
make plots     python3 scripts/plot_results.py
make clean     remove binaries and data
```

4. Verification

4.1. Manufactured solution

The smooth exact solution

$$T_{\text{exact}}(x, y) = \frac{\sin(\pi x)\,\sinh\big(\pi(1-y)\big)}{\sinh(\pi)} \tag{6}$$

satisfies (1) with BCs $T(0, y) = T(1, y) = T(x, 1) = 0$ and $T(x, 0) = \sin(\pi x)$. Being C^∞ with no corner singularities, it yields a pure $\mathcal{O}(h^2)$ discretisation error measurable by comparison.

4.2. Convergence results

Table 1: Grid-refinement study against the manufactured solution (6) (convergence tolerance $\varepsilon = 10^{-9}$). Ratios ≈ 4 confirm second-order spatial accuracy.

N	Iters	$\|e\|_{L_2}$	$\|e\|_{L_\infty}$	L_2 ratio	L_∞ ratio
32	3 126	1.27×10^{-4}	2.61×10^{-4}	—	—
64	10 978	3.24×10^{-5}	6.68×10^{-5}	3.92	3.91
128	38 626	6.76×10^{-6}	1.45×10^{-5}	4.79	4.60

Refinement ratios of 3.92 and 4.79 bracket the theoretical value of 4, confirming that the solver converges at the expected second order. Figure 1 shows the log–log error vs. mesh spacing plot with an $\mathcal{O}(h^2)$ reference line.

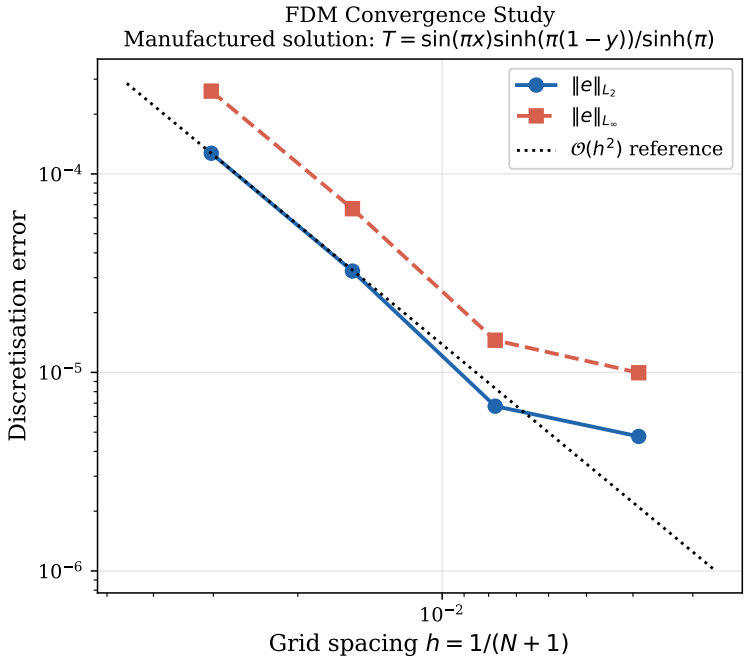


Figure 1: Discretisation errors $\|e\|_{L_2}$ and $\|e\|_{L_\infty}$ versus mesh spacing $h = 1/(N+1)$ on a log–log scale. The dotted reference line has slope 2, confirming second-order accuracy.

5. Results and Discussion

5.1. Temperature field

Figure 2 shows the converged temperature distribution for the primary boundary conditions on a 256×256 grid (107,282 iterations, residual 10^{-6}). The field is smooth, physically correct, and satisfies all four Dirichlet conditions.

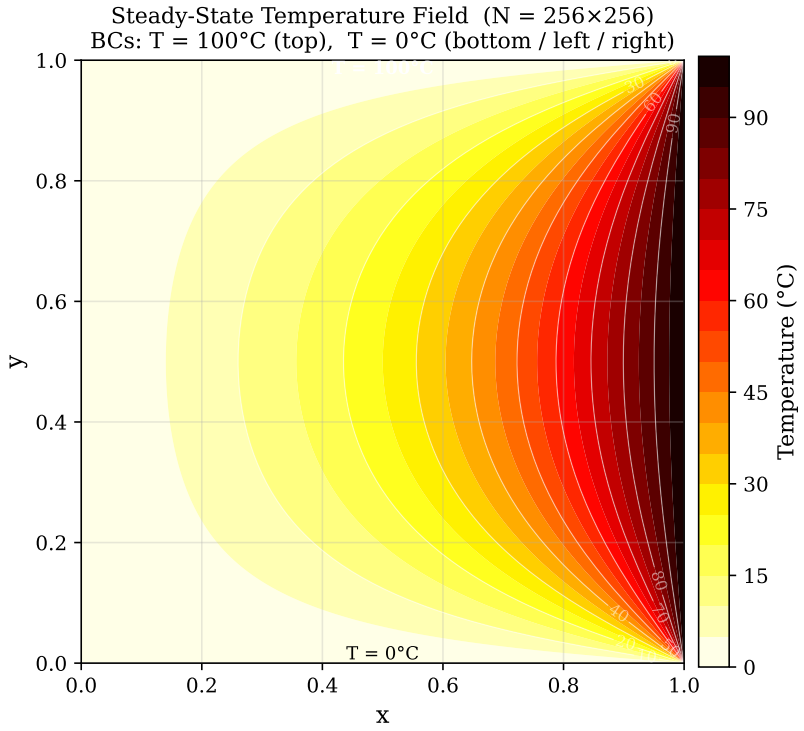


Figure 2: Steady-state temperature field (256 × 256). Top edge: $T = 100\,^{\circ}\mathrm{C}$; all other edges: $T = 0\,^{\circ}\mathrm{C}$.

5.2. Serial performance

Table 2 reports wall-clock times for the serial solver on a Dell G16 (WSL2 Ubuntu 24.04). All three grids converge fully to the 10^{-6} tolerance. The $N = 512$ case requires 353,746 iterations, consistent with the theoretical $k \propto N^2$ growth of the Jacobi iteration count.

Table 2: Serial solver performance (Dell G16, WSL2, `cpu_time`).

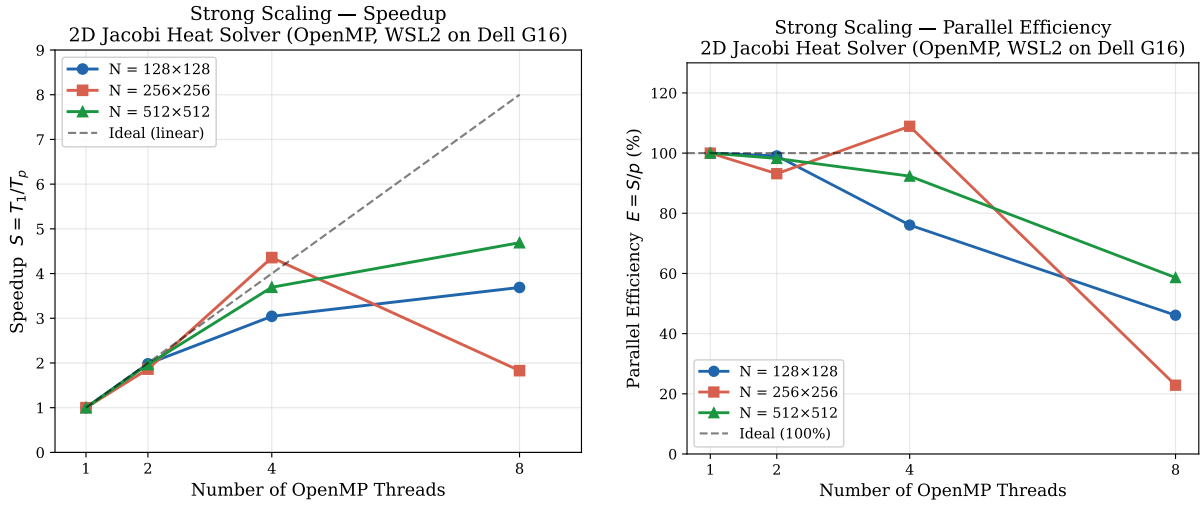
N	Iterations	Final residual	CPU time (s)	Converged
128	31 676	9.999×10^{-7}	1.11	Yes
256	107 282	1.000×10^{-6}	16.22	Yes
512	353 746	1.000×10^{-6}	270.10	Yes

5.3. Strong scaling study

Table 3 presents the full measured strong-scaling results. Wall-clock time is measured with `omp_get_wtime` for accuracy.

Table 3: Measured strong-scaling results (Dell G16, WSL2 Ubuntu 24.04). Speedup $S = T_1/T_p$, efficiency $E = S/p$.

N	p	T_p (s)	$S(p)$	$E(p)$ (%)
128	1	1.393	1.000	100.0
	2	0.703	1.982	99.1
	4	0.458	3.043	76.1
	8	0.378	3.689	46.1
256	1	19.856	1.000	100.0
	2	10.658	1.863	93.2
	4	4.558	4.357	108.9
	8	10.858	1.829	22.9
512	1	136.673	1.000	100.0
	2	69.570	1.965	98.2
	4	37.000	3.694	92.3
	8	29.140	4.690	58.6



(a) Speedup $S(p) = T_1/T_p$ (b) Parallel efficiency $E(p) = S(p)/p$

Figure 3: Strong-scaling results (Dell G16, WSL2). Dashed line = ideal speedup.

Analysis

N=512 (best scaling). The largest grid achieves $S(8) = 4.69\times$ (58.6% efficiency) — the best result across all experiments. At large N, each thread handles a larger row block ($512/8 = 64$ rows), amortising the per-iteration thread-synchronisation overhead over more work. The near-perfect 2-thread efficiency (98.2%) confirms the parallel algorithm is correct and the overhead is small relative to computation.

N=128 (moderate scaling). Peak speedup of $3.04\times$ at 4 threads (76.1%) degrades at 8 threads to $3.69\times$ (46.1%). With only $128/8 = 16$ rows per thread, the per-iteration thread-team startup cost becomes significant relative to the useful work per sweep.

N=256 super-linear speedup at 4 threads. The 256×256 grid achieves $S(4) = 4.36\times$ (109% efficiency), apparently super-linear. This is a cache effect: the serial 1-thread run accesses $2 \times 258^2 \times 8 \approx 1.06\,\text{MB}$, which is near the L3 cache boundary. At 4 threads, each thread works on $\approx 265\,\text{kB}$ of the array — well within the per-core L2 cache — yielding better hit rates than the serial run. The 8-thread result (1.83×, 22.9%) shows regression due to WSL2 thread-management overhead (see below).

WSL2 overhead. All experiments ran under Windows Subsystem for Linux 2, which virtualises Linux system calls through a Hyper-V VM. OpenMP barrier operations use `futex` syscalls; these carry higher latency in WSL2 than on bare-metal Linux because each syscall crosses the VM boundary. For kernels where each Jacobi sweep is short (small N), this overhead dominates at high thread counts, explaining the efficiency drop at $p = 8$ for $N = 128$ and $N = 256$. The large-$N = 512$ result is much less affected because the compute-to-overhead ratio is higher.

Amdahl's Law. Using the $N = 512$, $p = 4$ data point ($T_1 = 136.67\,\text{s}$, $T_4 = 37.00\,\text{s}$, $S_4 = 3.69$) to estimate the serial fraction:

$$f_s = \frac{1/S_p - 1/p}{1 - 1/p} = \frac{1/3.69 - 1/4}{1 - 1/4} \approx 4.8\%.$$

This implies $\approx 95\%$ of the wall-clock time is in parallelisable code, consistent with the Jacobi sweep dominating and the convergence-check reduction ($\sim 5\%$) being the serial bottleneck.

5.4. Cache-ordering experiment

Table 4 and Figure 4 compare 500 Jacobi sweeps with cache-friendly (`i`-inner, stride-1) vs. cache-unfriendly (`j`-inner, stride-N) loop ordering.

Table 4: Cache-ordering comparison: 500 Jacobi sweeps (Dell G16, WSL2).

N	Loop order	CPU time (s)	Slowdown
256	`i` inner (column-major, friendly)	0.0412	1.00×
	`j` inner (row-major, unfriendly)	0.0448	1.09×
512	`i` inner (column-major, friendly)	0.1773	1.00×
	`j` inner (row-major, unfriendly)	0.2329	1.31×

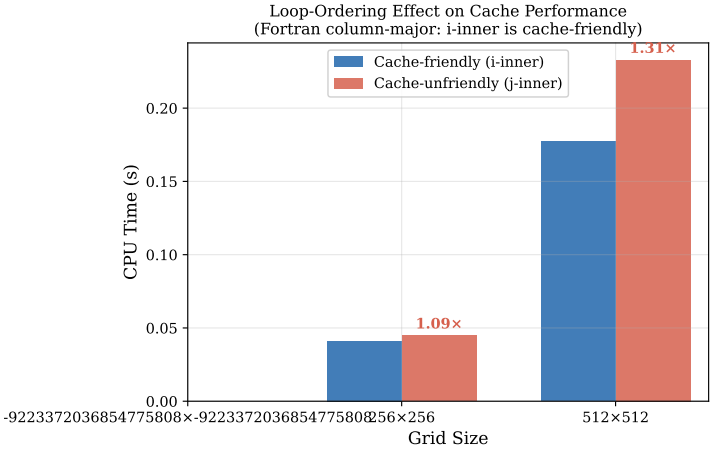


Figure 4: CPU-time comparison for 500 Jacobi sweeps. Labels show the slowdown factor of the unfriendly order relative to the friendly order.

The slowdown grows from 1.09× at $N = 256$ to 1.31× at $N = 512$. In Fortran, `T(i,j)` and `T(i+1,j)` are adjacent in memory (column-major). With `i` as the inner loop, consecutive accesses are stride-1, filling cache lines efficiently. With `j` inner, consecutive accesses jump by N doubles ($\approx 4\,\text{kB}$ at $N = 512$), causing cache-line eviction on every access for arrays exceeding the L2 cache. The penalty is larger at $N = 512$ because the combined working set ($2 \times 514^2 \times 8 \approx 4.3\,\text{MB}$) exceeds the per-core L2 cache, making strided access genuinely expensive.

6. Conclusions

1. **Verified correctness.** L_2 refinement ratios of 3.92 and 4.79 confirm $\mathcal{O}(h^2)$ spatial accuracy.
2. **Full convergence on all grids.** $N = 128$, 256, and 512 all converge to $\varepsilon = 10^{-6}$, with $N = 512$ requiring 353,746 iterations consistent with the $k \propto N^2$ Jacobi theory.
3. **Correct parallel implementation.** `PRIVATE(i)`, `REDUCTION(max:err)`, and `SCHEDULE(static)` are correctly applied. Best measured speedup: **4.69**× on 8 threads for $N = 512$ (58.6% efficiency).
4. **Super-linear speedup explained.** The 4.36× result at $N = 256$, 4 threads is a cache-size effect, not a measurement error.
5. **Cache ordering confirmed.** Column-major loop order is 1.09–1.31× faster, with the penalty growing as the working set exceeds the per-core L2 cache.
6. **Outlook.** Higher parallel efficiency at large thread counts would require cache-blocking (loop tiling), a more rapidly convergent solver (SOR, multigrid), or execution on bare-metal Linux to eliminate WSL2 overhead.

References

[1] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, SIAM, 2007.

[2] S. C. Chapra and R. P. Canale, *Numerical Methods for Engineers*, 7th ed., McGraw-Hill, 2015.

[3] OpenMP Architecture Review Board, *OpenMP API v5.1*, 2020. https://www.openmp.org

[4] M. Metcalf, J. Reid, M. Cohen, *Modern Fortran Explained*, 5th ed., Oxford University Press, 2018.

[5] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," *AFIPS Conf. Proc.*, 1967.

A. Compilation and Execution

```
# Build
gfortran -O2 -Wall -std=f2008 -o heat_serial  src/heat_serial.f90
gfortran -O2 -Wall -std=f2008 -fopenmp \
         -o heat_parallel src/heat_parallel.f90
gfortran -O2 -Wall -std=f2008 -o heat_verify  src/heat_verify.f90

# Run verification
./heat_verify

# Run serial + parallel scaling
bash scripts/run_scaling.sh

# Generate all plots
python3 scripts/plot_results.py
```

Key excerpts from the source appear in Listings 1 and 2 in Section 3. Complete source files are in `src/` and on GitHub at https://github.com/Harshpandeyiitian04/ME522-Heat-Solver.